Autenticación de Choferes — App Móvil TTPN

Fecha: 2026-04-11
Contexto: Propuesta de JWT para choferes (Employees) en la app móvil

---

Flujo actual (PHP — cómo funciona hoy)

El chofer no ingresa email ni contraseña personal. Ingresa la **clave de la unidad** que va a operar.

Paso 1 — Identificación de la unidad

```
Input del chofer: clave del vehículo  (ej. "T035")
```

El sistema busca el vehículo en `vehicles` por el campo `clv`.
Valida la contraseña: el formato es `ttpn` + número después del cero.

```
clv = "T035"  →  password = "ttpn35"
clv = "T001"  →  password = "ttpn1"
clv = "T022"  →  password = "ttpn22"
```

La contraseña está guardada **en texto plano** en `vehicles.password`.

Paso 2 — Manejo de la asignación actual

Si la unidad ya tiene un chofer asignado (`vehicle_asignations` con `fecha_hasta = NULL`):

- Se cierra la asignación actual: `UPDATE vehicle_asignations SET fecha_hasta = NOW()`
- Esto libera la unidad sin necesidad de intervención del chofer anterior.

Paso 3 — Validación del nuevo chofer

El sistema pide al chofer que ingrese sus datos personales:

Campo	Fuente
Nombre(s)	`employees.nombre`
Apellido Paterno	`employees.apaterno`
Fecha de Nacimiento	`employees.fecha_nacimiento`

El sistema busca un `Employee` que:

1. Coincida con los tres campos (nombre, apaterno, fecha_nacimiento)
2. Tenga `status = true` (activo)
3. Su último movimiento sea de tipo **Alta** (id=1) o **Reingreso** (id=3)

Si el match es exitoso → se crea la nueva asignación:

```
INSERT INTO vehicle_asignations (vehicle_id, employee_id, fecha_efectiva,
business_unit_id)
VALUES (?, ?, NOW(), ?)
```

Problemas del flujo actual

Problema	Impacto
No hay token — cada request valida la sesión localmente en PHP	Si el servidor reinicia, se pierde el estado
No hay forma de revocar el acceso de un chofer sin tocar el PHP	Seguridad
Los datos de sesión se guardan en `SharedPreferences` del Android sin cifrado	Riesgo si el teléfono se roba
Sin expiración: el chofer puede seguir usando la app indefinidamente	Compliance
Sin binding de dispositivo: el token podría usarse desde otro teléfono	Seguridad

---

Propuesta: JWT para Choferes

Concepto central

Los choferes NO son `Users` en el sistema — son `Employees`.
El JWT del chofer es **separado e incompatible** con el JWT de usuarios admin.

```
JWT de Usuario   →  payload: { user_id:, jti: }   →  valida en
BaseController
JWT de Chofer    →  payload: { employee_id:, vehicle_id:, jti: }  →
valida en MobileBaseController
```

Esto garantiza que un chofer **nunca** puede acceder a endpoints del panel admin, aunque intente usar su token.

Flujo propuesto en Rails

Endpoint de login

```
POST /api/v1/mobile/auth/login
```

Params:

```
{
  "vehicle_clv": "T035",
  "vehicle_password": "ttpn35",
  "nombre": "ANTONIO",
  "apaterno": "CASTELLANOS",
  "fecha_nacimiento": "1990-05-24"
}
```

Secuencia de validación

```
1. Buscar vehicle por clv
   └─ No encontrado → 401 "Unidad no encontrada"

2. Validar vehicle.password == vehicle_password
   └─ No coincide → 401 "Contraseña de unidad incorrecta"

3. Cerrar asignación actual (si existe)
   └─ VehicleAsignation con fecha_hasta: nil para este vehicle_id
   └─ UPDATE fecha_hasta = NOW()

4. Buscar Employee por nombre + apaterno + fecha_nacimiento
   └─ No encontrado → 422 "Datos del chofer no coinciden"

5. Validar employee.status == true
   └─ Falso → 403 "Empleado inactivo"

6. Validar último movimiento = Alta (1) o Reingreso (3)
   └─ Es Baja o Baja-Lista Negra → 403 "Empleado dado de baja"

7. Crear nueva VehicleAsignation (employee ↔ vehicle)

8. Generar JWT del chofer
   └─ payload: { employee_id:, vehicle_id:, jti: UUID, exp: 7.días }

9. Responder con token + datos del chofer
```

Implementación técnica

Migración: agregar `jti` a `employees`

```ruby
# db/migrate/YYYYMMDD_add_jti_to_employees.rb
class AddJtiToEmployees < ActiveRecord::Migration[7.1]
  def change
    add_column :employees, :jti, :string
    add_index :employees, :jti, unique: true
```

```
  end
end
```

Método JWT en el modelo Employee

```
# app/models/employee.rb

# Genera un nuevo JWT y guarda el jti en BD (invalidando el anterior)
def generate_driver_jwt
  new_jti = SecureRandom.uuid
  update_column(:jti, new_jti)

  payload = {
    employee_id: id,
    vehicle_id:  current_vehicle_id,
    jti:         new_jti,
    exp:         7.days.from_now.to_i
  }

  JWT.encode(payload, Rails.application.credentials.secret_key_base,
'HS256')
end

# Revoca el token actual (logout del chofer)
def revoke_driver_jwt!
  update_column(:jti, SecureRandom.uuid)
end

# Obtiene el vehicle_id actual según la asignación vigente
def current_vehicle_id
  vehicle_asignations.where(fecha_hasta: nil).order(fecha_efectiva:
:desc).first&.vehicle_id
end

# Valida si el empleado puede operar (activo + último movimiento
Alta/Reingreso)
def elegible_para_operar?
  return false unless status

  ultimo_mov = employee_movements
    .order(fecha_efectiva: :desc)
    .first
  return false unless ultimo_mov

  EmployeeMovementType.altas_y_reingresos_ids.include?
(ultimo_mov.employee_movement_type_id)
end
```

Controlador de autenticación móvil

```ruby
# app/controllers/api/v1/mobile/auth_controller.rb

module Api
  module V1
    module Mobile
      class AuthController < ActionController::API
        skip_before_action :verify_authenticity_token, raise: false

        # POST /api/v1/mobile/auth/login
        def login
          # Paso 1: Identificar vehículo
          vehicle = Vehicle.find_by(clv: params[:vehicle_clv])
          unless vehicle
            return render json: { error: 'Unidad no encontrada' }, status:
:unauthorized
          end

          unless vehicle.password == params[:vehicle_password]
            return render json: { error: 'Contraseña de unidad incorrecta'
}, status: :unauthorized
          end

          # Paso 2: Cerrar asignación vigente (si existe)
          asignacion_actual = VehicleAsignation.where(vehicle_id:
vehicle.id, fecha_hasta: nil).first
          asignacion_actual&.update_column(:fecha_hasta, Time.current)

          # Paso 3: Buscar empleado por datos personales
          employee = Employee.find_by(
            nombre:           params[:nombre].to_s.upcase.strip,
            apaterno:         params[:apaterno].to_s.upcase.strip,
            fecha_nacimiento: params[:fecha_nacimiento]
          )

          unless employee
            return render json: { error: 'Datos del chofer no coinciden
con ningún empleado' }, status: :unprocessable_entity
          end

          # Paso 4: Validar elegibilidad
          unless employee.elegible_para_operar?
            return render json: { error: 'Empleado inactivo o dado de
baja' }, status: :forbidden
          end

          # Paso 5: Crear nueva asignación
          VehicleAsignation.create!(
            vehicle_id:      vehicle.id,
            employee_id:     employee.id,
            fecha_efectiva:  Time.current,
            business_unit_id: vehicle.business_unit_id ||
employee.business_unit_id
          )
```

```
          # Paso 6: Emitir JWT del chofer
          token = employee.generate_driver_jwt

          render json: {
            access_token: token,
            token_type:   'Bearer',
            expires_in:   7.days.to_i,
            chofer: {
              id:               employee.id,
              clv:              employee.clv,
              nombre:           employee.nombre,
              apaterno:         employee.apaterno,
              amaterno:         employee.amaterno,
              business_unit_id: employee.business_unit_id
            },
            vehiculo: {
              id:     vehicle.id,
              clv:    vehicle.clv,
              modelo: vehicle.modelo,
              placa:  vehicle.placa
            }
          }
        end

        # DELETE /api/v1/mobile/auth/logout
        def logout
          # Necesita autenticación previa — ver MobileBaseController
          current_employee&.revoke_driver_jwt!
          render json: { message: 'Sesión cerrada' }
        end
      end
    end
  end
end
```

BaseController para endpoints móviles

```
# app/controllers/api/v1/mobile_base_controller.rb

class Api::V1::MobileBaseController < ActionController::API
  before_action :authenticate_driver

  private

  def authenticate_driver
    auth = request.headers['Authorization']
    return unauthorized! unless auth&.start_with?('Bearer ')

    token = auth.split(' ', 2).last
    begin
```

```
      decoded = JWT.decode(
        token,
        Rails.application.credentials.secret_key_base,
        true,
        algorithm: 'HS256'
      ).first

      employee = Employee.find(decoded['employee_id'])

      # Validar JTI (revocación)
      if decoded['jti'] != employee.jti
        return render json: { error: 'Sesión revocada. Ingresa
nuevamente.' }, status: :unauthorized
      end

      @current_employee = employee
      @current_vehicle_id = decoded['vehicle_id']

    rescue JWT::ExpiredSignature
      render json: { error: 'Sesión expirada. Vuelve a asignarte la
unidad.' }, status: :unauthorized
    rescue JWT::DecodeError, ActiveRecord::RecordNotFound
      render json: { error: 'Token inválido' }, status: :unauthorized
    end
  end

  def current_employee
    @current_employee
  end

  def current_vehicle_id
    @current_vehicle_id
  end

  def unauthorized!
    render json: { error: 'No autenticado' }, status: :unauthorized
  end
end
```

Rutas

```
# config/routes.rb

namespace :api do
  namespace :v1 do
    # Auth de usuarios admin (ya existe)
    namespace :auth do
      post   'login',  to: 'sessions#create'
      delete 'logout', to: 'sessions#destroy'
      get    'me',     to: 'sessions#me'
    end
```

```
    # Auth de choferes (nuevo)
    namespace :mobile do
      post   'auth/login',  to: 'auth#login'
      delete 'auth/logout', to: 'auth#logout'

      # Endpoints móviles protegidos con JWT de chofer
      resources :travel_counts, only: [:index, :create]
      resources :bookings,      only: [:index, :show]
      resources :gas_charges,   only: [:index, :create]
      resources :vehicle_checks, only: [:index, :create]
    end
  end
end
```

Ejemplo de controller móvil protegido

```
# app/controllers/api/v1/mobile/travel_counts_controller.rb

module Api
  module V1
    module Mobile
      class TravelCountsController < Api::V1::MobileBaseController
        # current_employee y current_vehicle_id disponibles aquí

        def create
          tc = TravelCount.new(travel_count_params)
          tc.employee_id = current_employee.id
          tc.vehicle_id  = current_vehicle_id

          if tc.save
            render json: tc, status: :created
          else
            render json: { errors: tc.errors.full_messages }, status:
:unprocessable_entity
          end
        end
      end
    end
  end
end
```

Comparación: flujo actual vs propuesto

Aspecto	PHP actual	Rails + JWT
Identificación del chofer	Datos personales (no contraseña)	Igual — nombre + apaterno + fecha_nac

Aspecto	PHP actual	Rails + JWT
Identificación de la unidad	`clv` + `ttpnXX` (texto plano)	Igual
Token de sesión	Ninguno (estado en SharedPreferences)	JWT firmado con HMAC-SHA256
Expiración	Nunca (hasta logout manual)	7 días
Revocación	Imposible sin reiniciar PHP	Inmediata vía JTI
Separación admin/chofer	Ninguna — mismos endpoints	Namespaces separados + JWTs distintos
Acceso a datos de admin	Posible si se conoce el endpoint	Imposible — JTI diferente

Seguridad: consideraciones adicionales

Normalización de nombres (crítica)

Los nombres en la BD están en MAYÚSCULAS. La app debe normalizar antes de comparar:

```
nombre:   params[:nombre].to_s.upcase.strip,
apaterno: params[:apaterno].to_s.upcase.strip
```

Considerar también ignorar acentos con `.unicode_normalize(:nfd).gsub(/\p{Mn}/, '')`.

Binding por dispositivo — IMEI descartado ❌

`Employee` tiene campo `imei` (legacy del PHP), pero **obtener el IMEI en Android es imposible desde Android 10+**: Google bloqueó el acceso a `TelephonyManager.getDeviceId()` sin permiso de nivel sistema. Intentado y descartado.

Alternativa viable: Android ID

`Settings.Secure.ANDROID_ID` es un identificador único por dispositivo + app + usuario. No es el IMEI pero cumple el mismo propósito para binding:

```
// Android (Kotlin)
val androidId = Settings.Secure.getString(
    context.contentResolver,
    Settings.Secure.ANDROID_ID
)
// Se envía en el header: X-Device-ID: <androidId>
```

```
# Rails — MobileBaseController
# Opcionalmente validar que el device_id del request coincide con el
registrado
device_id = request.headers['X-Device-ID']
if employee.imei.present? && device_id != employee.imei
  return render json: { error: 'Dispositivo no autorizado' }, status:
:unauthorized
end
# Al primer login exitoso, guardar el device_id:
employee.update_column(:imei, device_id) if employee.imei.blank?
```

Limitaciones del Android ID:

- Cambia si el usuario hace factory reset
- Cambia si el usuario reinstala la app en algunos fabricantes
- En PWA/browser no existe este API — se necesitaría fingerprint del browser (menos confiable)

Decisión: Para TTPN, con choferes en dispositivos corporativos, el Android ID es suficiente. Para PWA, omitir el binding de dispositivo y confiar en el JTI + expiración de 7 días.

Rate limiting en login

El endpoint de login acepta datos personales — agregar rate limiting con `Rack::Attack` o Nginx para evitar enumeración de choferes:

```
# config/initializers/rack_attack.rb
Rack::Attack.throttle('mobile_login', limit: 5, period: 60) do |req|
  req.ip if req.path == '/api/v1/mobile/auth/login' && req.post?
end
```

Duración del token

Opción	Pros	Contras
24 horas	Fuerza re-asignación diaria (auditoría)	Incómodo si hay turnos largos
7 días	Balance entre seguridad y UX	Ventana de riesgo mayor
Indefinido + logout	Sin fricción	Depende de que el chofer haga logout

Recomendación: 7 días con refresh implícito (si el token tiene menos de 1 día para expirar, el endpoint `/me` devuelve un nuevo token automáticamente).

Pendientes para implementar

Tarea	Prioridad
Migración `add_column :employees, :jti, :string`	Alta — bloquea todo lo demás

Tarea	Prioridad
Método `elegible_para_operar?` en Employee	Alta
Método `generate_driver_jwt` en Employee	Alta
`Api::V1::MobileBaseController`	Alta
`Api::V1::Mobile::AuthController` (login/logout)	Alta
Rutas `/api/v1/mobile/*`	Alta
`Api::V1::Mobile::TravelCountsController`	Alta — core de la app
`Api::V1::Mobile::BookingsController`	Alta
`Api::V1::Mobile::GasChargesController`	Media
`Api::V1::Mobile::VehicleChecksController`	Media
Normalización de acentos en búsqueda de nombre	Media
Rate limiting en login endpoint	Media
IMEI binding (opcional)	Baja
Endpoint `/api/v1/mobile/auth/refresh`	Baja

Relación con PLAN_MIGRACION_MOVIL.md

Este documento cubre el **Bloque A** del plan de migración (fase 2):

```
BE-1 → Endpoint POST /api/v1/mobile/auth/login  (este doc)
BE-2 → TravelCounts controller (MobileBaseController ya protege)
BE-3 → Bookings del chofer
BE-4 → GasCharges con GPS
BE-5 → VehicleChecks con foto
```

El chofer que hoy usa `Gasto_LOGIN.php` usaría exactamente el mismo flujo de ingreso — mismos campos, mismo proceso de asignación de unidad — solo que ahora con un JWT seguro de vuelta.